

TECHNICAL PROJECT REPORT

ALIGNHIRE

AI Talent Intelligence Platform

Frontend Prototype — Architecture, Feature & Technology Documentation



Fig. 0 — AlignHire solution overview: talent and hiring dashboards

Problem Statement

Existing manual/legacy processes in this domain are time-consuming, error-prone, and difficult to scale. Users currently rely on fragmented tools or manual effort to manage their tasks, leading to inefficiency, delays, and lack of real-time visibility. There is a clear need for a unified, easy-to-use software solution that addresses these gaps effectively.

Our Solution

We propose a software application that streamlines the entire workflow through automation, a simple user interface, and real-time data handling. Key highlights of our solution include:

- Centralized platform to manage tasks efficiently and reduce manual effort.
- Intuitive and user-friendly interface accessible across devices.
- Real-time processing and data synchronization for better decision-making.
- Scalable architecture designed to accommodate future growth and features.

Team Details

Role	Name
Team Leader	Mohana Priyan S
Team Member	Boomesh
Team Member	Hafiz Muzzamil

1. Executive Summary

AlignHire is an AI-oriented talent intelligence platform concept that connects job seekers and hiring teams through resume parsing, profile scoring, and job-candidate alignment. This report documents the project exactly as it exists in the submitted build: a fully designed, interactive front-end web application (single HTML/CSS/JavaScript file) that demonstrates the complete product experience for two user roles — Talent (job seeker) and Hiring Team (recruiter) — using simulated, client-side data.

The report is written to reflect the actual codebase and assets supplied for review (the project's index HTML file and the accompanying architecture/UI reference images). It separates clearly what is implemented and running in the current build from what is proposed as the target system architecture for a future production version. No claim in this document describes functionality that is not present in the reviewed files.

2. Project Overview

AlignHire's stated purpose is to reduce manual effort in hiring by automatically extracting structured information from resumes, comparing it against job requirements, and producing an “alignment score” that ranks how well a candidate fits a role. The product is designed around two dashboards:

- Talent Dashboard (Job Seeker) — lets a candidate upload a resume, view a profile-strength score, see top matched jobs, track applications, and review career analytics.
- Hiring Team Dashboard (Recruiter) — gives recruiters an overview of open jobs, applicant volume, shortlisted/hired counts, a ranked list of top candidates, and monitoring of the underlying “AI agents” that would perform extraction, parsing, matching, and alignment in a live system.

In the current build, both dashboards are fully navigable and visually complete, and all figures, charts, and “AI” outputs (skills, scores, match percentages, timelines) are pre-set demonstration values rendered on the client side — they are not produced by a live machine-learning pipeline.

3. Proposed System Architecture (Design Target)

The reference architecture diagram supplied with the project (Fig. 1) lays out a five-stage data-and-AI pipeline that describes the intended end-to-end system once a backend and models are built. This pipeline is the design blueprint for the product; it is not yet implemented in the reviewed codebase, which currently contains front-end code only.

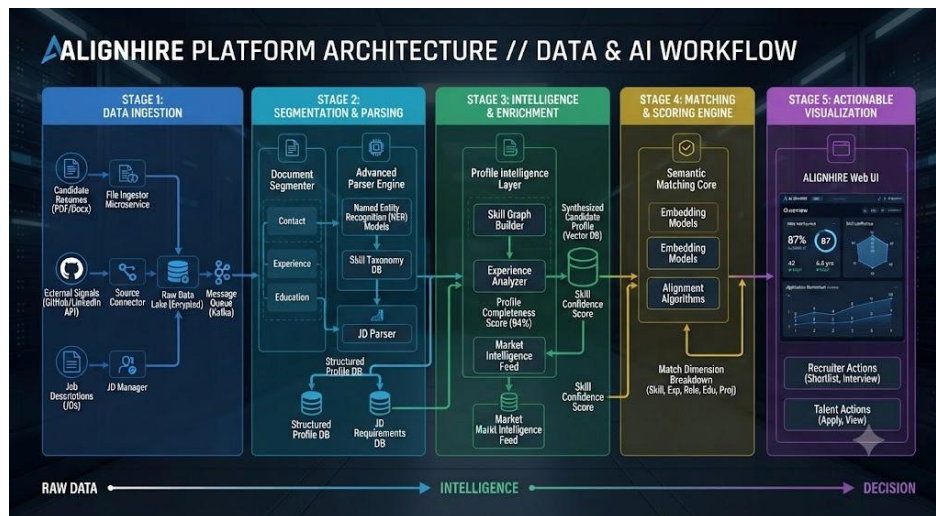


Fig. 1 — AlignHire platform architecture: data & AI workflow (design reference)

3.1 Stage 1 — Data Ingestion

Intended to accept candidate resumes (PDF/DOCX) and job descriptions through a file microservice, alongside external signals (e.g., GitHub/LinkedIn APIs) via a source connector, feeding an encrypted raw data lake and a message queue for downstream processing.

3.2 Stage 2 — Segmentation & Parsing

Planned document segmentation (contact, experience, education) followed by an advanced parser using Named Entity Recognition (NER) models and a skill-taxonomy database, producing a structured profile database and a job-description requirements database.

3.3 Stage 3 — Intelligence & Enrichment

A proposed profile intelligence layer combining a skill-graph builder, experience analyzer, and market-intelligence feed to generate a completeness score and a synthesized candidate profile stored as vector embeddings, alongside a skill-confidence score.

3.4 Stage 4 — Matching & Scoring Engine

A semantic matching core using embedding models and alignment algorithms to compare candidate and job vectors, producing a match-dimension breakdown across skills, experience, role, education, and projects.

3.5 Stage 5 — Actionable Visualization

The scored, matched output is intended to surface in the AlignHire web UI — the same dashboards documented in Section 5 — driving recruiter actions (shortlist, interview) and talent actions (apply, view).

Because the supplied project files contain only the presentation layer, Stages 1–4 above should be read as the system's intended roadmap rather than delivered functionality; Stage 5 (the UI) is the part that has been built and is reviewed in detail in this report.

4. Current Implementation Status

The submitted codebase consists of a single self-contained HTML document that renders a multi-view single-page application (SPA) entirely in the browser. There is no server-side code, no database connection, and no external API call in the file. This section states plainly what runs today.

Layer	Status	Notes
User Interface (both dashboards)	Implemented	Fully built, styled, and interactive
Client-side navigation / routing	Implemented	JavaScript view-switching, no page reloads
Charts & analytics visuals	Implemented	Rendered with Chart.js from static demo data
Resume upload UI	Implemented (UI only)	File picker accepts .pdf / .docx; no parsing occurs
Resume parsing / NER	Not implemented	Simulated with a timed loading state and fixed sample output
Matching & alignment scoring	implemented	Scores are hard-coded demonstration values
AI agents (extract, segment, parse, match, align)	implemented	Represented as static status indicators, not running services
Backend services / database / vector store	implemented	No such code is present in the reviewed files

Table 1 — Implementation status of AlignHire, verified against the submitted files

5. Frontend Features

The interface is organized as a single-page application with a persistent sidebar and a role switch (Talent / Hiring). The features below are all present and navigable in the reviewed build.

5.1 Talent (Job Seeker) Experience

- Overview dashboard showing profile summary, contact details, and an “overall alignment” ring score.
- Top-skills panel with per-skill proficiency bars (e.g., JavaScript, React, Node.js, MongoDB, HTML/CSS).
- “Top matched jobs” list with per-job match percentage and a link to view details.
- Resume Parser Cockpit — a drag-and-drop resume upload zone, a target job-description text box, a parser-mode selector, and a simulated pipeline that produces a structured candidate profile and a fit-score breakdown.
- Application Tracker — a kanban-style view of applications across stages such as Applied, Shortlisted, and Interview.
- Career Analytics — line and radar charts for application momentum, responses, and skill confidence, plus a search funnel and recommended next actions.
- Job Preferences — toggles for remote-role prioritization, a weekly digest, and a target compensation range.
- Match Detail view — a dimension-by-dimension breakdown (skills, experience, role, education, projects) explaining why a candidate matched a specific job.

5.2 Hiring Team (Recruiter) Experience

- Job overview cards — active jobs, total applicants, shortlisted count, and hires.
- Top-candidates list ranked by alignment percentage, with role and status.
- Applicants-over-time chart showing pipeline growth.
- AI Agent Monitoring panel — status indicators for the Extraction, Parsing, Matching, and Alignment agents (presented as “ready/active” states, not live processes).
- Job creation form — fields for title, department, location, and a pasted job description.
- Talent Intelligence Grid — a searchable, sortable, filterable candidate directory (search by name/role/skill; sort by alignment, recency, or experience).
- Candidate detail modal with a recruiter note field.
- Cohort analytics, shortlist pipeline, and an “Autonomous Agent Operations” screen with a live-looking activity log stream.
- Exportable hiring report action on the pipeline-health view.

5.3 Shared Platform Features

- Global command palette (search talent, jobs, and skills from a single keyboard-driven modal).
- Notification dropdown, theme toggle (light/dark), and user profile menu.
- Toast notifications for background actions (e.g., “Resume uploaded – read-only; grounded parsing rules applied”).
- Responsive layout with a collapsible sidebar for smaller screens.

6. Technologies & Tools Used

This section lists only the technologies that are verifiably present in the submitted source file, followed separately by the technologies named in the architecture diagram for the proposed (not yet built) backend and AI layer.

6.1 Implemented — Frontend Build

Category	Technology	Purpose in this project
Markup / structure	HTML5	Single-page application shell
Styling	Tailwind CSS (CDN build) + custom CSS	Utility-first styling, dark theme, glassmorphism panels
Logic / interactivity	Vanilla JavaScript	View routing, state object, event handlers, simulated pipelines
Charts	Chart.js	Application momentum, skill confidence, pipeline, funnel charts
Icons	Lucide Icons	UI iconography throughout both dashboards
Typography	Google Fonts — Manrope, DM Sans, JetBrains Mono	Display, body, and monospace type

Table 2 — Technologies confirmed by direct inspection of the source file

6.2 Proposed — Backend & AI Layer (Per Architecture Diagram)

These items appear only in the architecture reference diagram (Fig. 1) as the intended technical approach for a production system. They are documented here for completeness and clearly marked as not yet built.

Pipeline stage	Proposed components
Data ingestion	File ingestor microservice, source connector, encrypted raw data lake, Kafka message queue
Segmentation & parsing	Document segmenter, NER-based parser engine, skill taxonomy DB, JD parser
Intelligence & enrichment	Skill graph builder, experience analyzer, market intelligence feed, vector database
Matching & scoring	Embedding models, semantic matching core, alignment algorithms
Visualization	AlignHire web UI (the layer already implemented and reviewed in Section 5)

Table 3 — Proposed backend/AI stack, as named in the architecture diagram — design intent only

7. Data & Interaction Model (Current Build)

All screen content is driven by a single in-memory JavaScript state object defined at the top of the script. Actions such as uploading a resume or creating a job description trigger timed UI transitions (JavaScript setTimeout calls) that simulate processing delay before revealing pre-written result data — for example, a fixed 87% alignment score and a

fixed skills breakdown. The “live agent activity stream” on the Autonomous Agent Operations screen selects log lines at random from a fixed list to imitate real-time activity.

No network requests (fetch/XHR), no external AI API calls, and no persistent storage were found in the file. Refreshing the browser resets all data to its original demo state.

8. Application Screenshots



Fig. 2 — Talent dashboard overview: alignment score, AI agent status strip, application momentum and skill-confidence charts



Fig. 3 — Combined product view: talent dashboard, hiring dashboard, workflow, and key benefits

9. Known Limitations of the Current Build

- All alignment scores, skill percentages, and analytics values are static demonstration data, not computed from real resumes or job descriptions.
- Resume parsing and job matching are simulated with timed loading states; no NLP or machine-learning model runs in the browser.
- There is no backend, database, or authentication system in the reviewed files — the app is a client-only prototype.
- Data does not persist between sessions; refreshing the page reverts to default demo values.
- The “AI Agents” shown on-screen are UI status indicators, not autonomous running processes.

10. Recommended Next Steps

- Build the backend services described in Fig. 1 (ingestion API, parsing service, matching engine) and connect them to this existing frontend via REStapi endpoints.
- Replace simulated resume parsing with an actual document-parsing and NER pipeline, and persist structured profiles in a real database.
- Implement the embedding-based matching engine and a vector database to generate genuine alignment scores in place of the current fixed values.
- Add authentication, role-based access, and data persistence so candidate and recruiter sessions carry real state.
- Introduce automated testing and a CI/CD pipeline once backend services are added to the repository.

11. Conclusion

AlignHire, as submitted, is a well-designed and functionally complete front-end prototype for an AI talent-matching platform. It successfully demonstrates the intended user experience for both job seekers and recruiters using HTML, Tailwind CSS, vanilla JavaScript, and Chart.js. The AI-driven capabilities — resume parsing, profile enrichment, semantic matching, and alignment scoring — are represented in the interface and are clearly specified in the accompanying architecture diagram, but they remain a design target rather than delivered functionality in this build. Treating the current deliverable accurately as a frontend prototype, with the backend and AI pipeline as the next development phase, gives an honest baseline for planning the platform's remaining engineering work.